Running LLM Inference in Kubernetes

KCD HELSINKI 2026

Agenda

ACTS / SECTIONS

Act 1

This presentation.

20 min.

Act 2

Hands on.

60 min.

Act 3

What’s next? (llm-d) and Q&A.

25 min.

"Just put it behind a Service" doesn't work

LLM INFERENCE BREAKS EVERY DEFAULT KUBERNETES SERVING ASSUMPTION

HTTP vs LLM

An HTTP request typically consists of just a few **kilobytes**. In stark contrast, an LLM request can reach **gigabytes**, drastically increasing the resource demands on the system.

Memory Implications

The memory requirements for LLM requests create unique challenges for deployment. Managing these demands effectively is crucial to ensure optimal performance and system stability under load.

An LLM Request

THE SIZE OF DATA REQUESTS

Two phases per request: **prefill** (compute-bound) → **decode** (memory-bound, autoregressive).

Each request carries a growing KV cache

TTFT measures prefill. **Throughput** measures decode. They have different bottlenecks.

Output length is unknown in advance.

vLLM

OPEN-SOURCE LLM SERVING ENGINE

WHAT

Single process. One model. One (or a few) GPUs. OpenAI-compatible API

WHY

Naive serving wasted 60–80% of GPU memory. vLLM solved it with PagedAttention.

WHERE

vLLM is the engine. The "Production Stack" is the Helm chart we wrap around it.

Page the KV cache like you'd page RAM

60–80% WASTE → ~4% WASTE

BEFORE

Each sequence pre-reserved a contiguous KV buffer at max length. 60–80% of KV memory wasted to internal + external fragmentation.

PagedAttention

Split KV into fixed-size blocks. Each sequence has a block table mapping logical positions → physical blocks. A page table on a GPU.

AFTER

Waste drops to ~4%. Continuous batching becomes possible — scheduler admits and evicts sequences at decode-step granularity.

Limitations of Single-GPU Deployment

Cross-replica routing

vLLM doesn't know about its siblings. You need a router in front.

Autoscaling

vLLM doesn't watch its queue. Replica count is the cluster's problem.

Multi-model on shared GPUs

One engine, one model. Multiple models = multiple pods, each holding weights.

Three projects worth knowing

PRODUCTION STACK IS TODAY'S WORKSHOP. OTHERS FOR CONTEXT.

vLLM Production Stack

Helm chart from the vLLM project — engine, router, observability hooks. Today's workshop. Reach for it: shortest path from helm install to a working OpenAI endpoint.

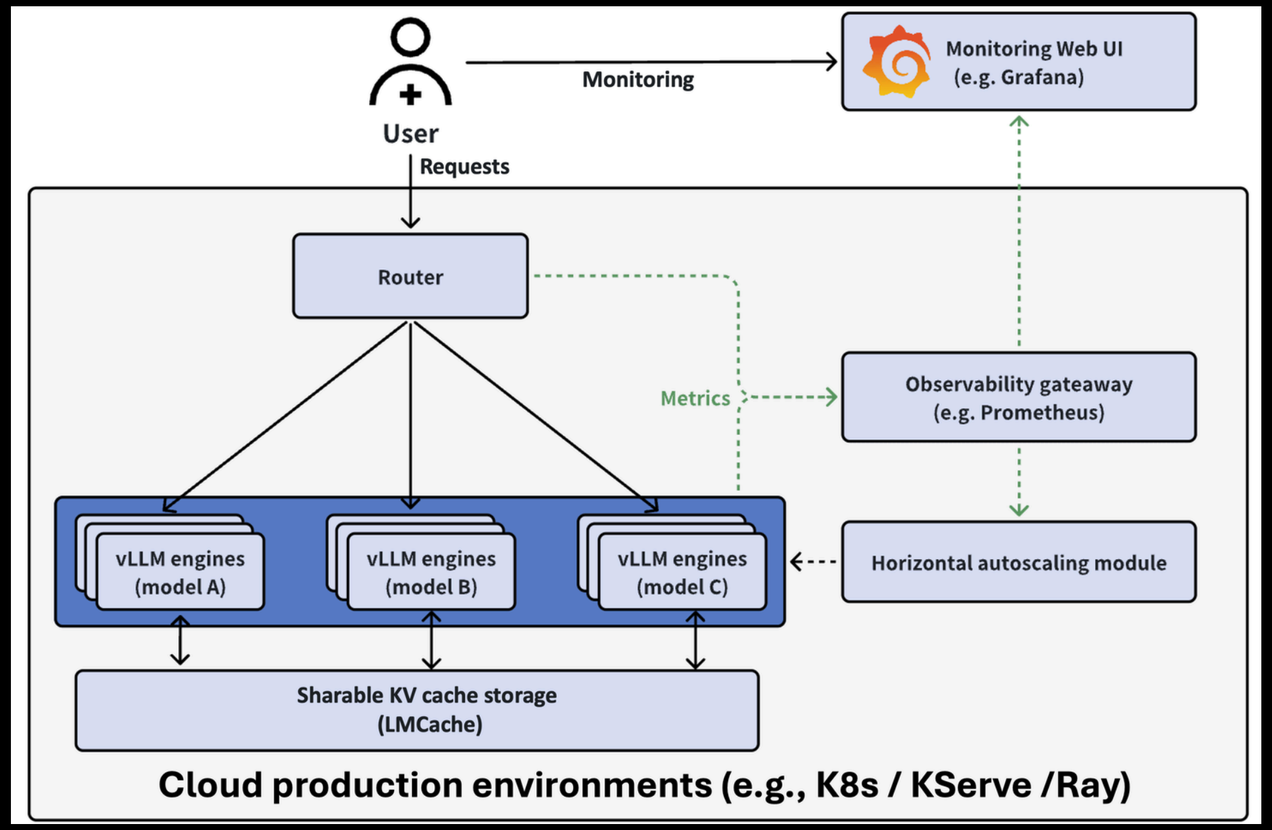
llm-d

CNCF Sandbox, pre-1.0. Disaggregated prefill/decode, KV-aware scheduling, multi-node. Reach for it: models bigger than one node, or TTFT-under-load is the SLA.

KubeAI

Operator with a Model CR; picks a backend. Scale-to-zero. Reach for it: multi-modality with one CRD across LLM, embeddings, STT.

Production Stack architecture



Two pods you own — router + engine. Two cluster-wide tenants — Prometheus + KEDA. The chart ships the top; the cluster operator installed the bottom before you walked in.

→ Use prefixaware, not roundrobin, for chat.
→ Scale on num_requests_waiting, not GPU utilisation.

Use Cases Overview

Chat

Multi-turn dialogue improves **latency and response rates**.

Batch

Burst processing stabilizes **throughput under load conditions**.

Agentic

Concurrency management enhances **request handling efficiency**.

Let's deploy

HANDS-ON COMMAND FOR YOUR WORKSHOP

Deployment command

To begin, use the command **export MY_NAMESPACE=group-XX**. Replace XX with your assigned group number to set up your environment correctly.

Follow-up commands

Execute the scripts **./scripts/01-deploy.sh** and **./scripts/02-port-forward.sh** in separate terminals to initiate the deployment and access the service effectively.